

Otulia Web: Deep Dive - Chapter 2: The Data Layer (MongoDB & Models)

2.1 Introduction to the Otulia Data Architecture

In a luxury marketplace, data integrity and granularity are paramount. Buyers need to know everything from the "Cylinder Capacity" of a car to the "Hull Material" of a yacht. This chapter explores how we use Mongoose and MongoDB to handle these complex requirements through a "Multi-Schema" architecture.

2.2 User Management & Relationships (`User.model.js`)

The User model is the backbone of the platform, managing identity, permissions, and history.

2.2.1 Core Identity & Security

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true }, // Removes leading/trailing spaces
  email: {
    type: String,
    required: true,
    unique: true, // Prevents duplicate accounts
    lowercase: true // Normalizes email for consistent login
  },
  password: { type: String, required: false, minlength: 6 }, // Optional for Google OAuth
  googleId: { type: String, unique: true, sparse: true }, // 'sparse' allows multiple nulls
  role: {
    type: String,
    enum: ["user", "agent", "admin"], // Strict role enforcement
    default: "user"
  },
});
```

2.2.2 The Tiered Subscription System

Otulia uses a "Freemium" model.

```
plan: {
  type: String,
  enum: ["Freemium", "Premium Basic", "Business VIP"],
  default: "Freemium"
},
planExpiresAt: { type: Date }, // Tracks when to downgrade the user
```

2.2.3 Polymorphic Histories (Advanced Mongoose)

This is one of the most powerful parts of the Otulia backend. Instead of separate arrays for car favorites and boat favorites, we use **refPath**.

```

favorites: [
  {
    assetId: {
      type: mongoose.Schema.Types.ObjectId,
      refPath: "favorites.assetModel" // Dynamic reference
    },
    assetModel: {
      type: String,
      enum: ["Listing", "CarAsset", "EstateAsset", "YachtAsset", "BikeAsset"]
    },
    addedAt: { type: Date, default: Date.now }
  }
],

```

Explanation: When you call `.populate('favorites.assetId')`, Mongoose looks at the `assetModel` field to decide which collection to join with. This allows a single array to hold any type of luxury asset.

2.3 Specialized Luxury Asset Models

Generic schemas aren't enough for high-end items. We maintain separate models for each category to capture specific technical metadata.

2.3.1 The Car Model (`CarAsset.model.js`)

Captures data crucial for automotive collectors.

```

const carAssetSchema = new mongoose.Schema({
  title: { type: String, required: true },
  price: { type: Number, required: true },
  isPriceOnRequest: { type: Boolean, default: false }, // For ultra-rare "P.O.A" items

  specification: {
    power: String, // e.g., "720 PS"
    topSpeed: String, // e.g., "341 km/h"
    matchingNumbers: String, // "Yes/No" - critical for vintage car value
    accidentHistory: String, // Transparency for high-value sales
    usageStatus: String // "Used" or "Unused"
  },

  agent: { // Embedded agent info for high-speed retrieval
    id: String,
    name: String,
    companyLogo: String
  },

  acquisition: { type: String, enum: ['rent', 'buy', 'rent/buy'] }
}, { timestamps: true });

```

2.3.2 The Yacht Model (`YachtAsset.model.js`)

Naval-specific metadata.

```
keySpecifications: {
  length: String, // e.g., "40 Meters"
  fuelCapacity: String, // e.g., "20,000 Liters"
  guestCapacity: String, // Essential for charter yachts
  hullMaterial: String // e.g., "Fiberglass" or "Steel"
},
```

2.3.3 The Estate Model (EstateAsset.model.js)

Architectural and property data.

```
specification: {
  architectureStyle: String, // e.g., "Modernist" or "Victorian"
  builtUpArea: String, // House size
  landArea: String, // Total plot size
  amenities: [{ type: String }], // e.g., "Infinity Pool", "Wine Cellar"
  smartHomeSystems: [{ type: String }] // Modern luxury requirement
},
```

2.4 Lead & Interaction Tracking

These models handle the "Business" side of the marketplace—connecting buyers to sellers.

2.4.1 Lead Management (Lead.model.js)

When a user "Inquires" about an asset, a Lead is created.

```
const leadSchema = new mongoose.Schema({
  sender: { type: mongoose.Schema.Types.ObjectId, ref: "User" }, // The Buyer
  agentId: { type: mongoose.Schema.Types.ObjectId, ref: "User" }, // The Seller
  assetId: { type: mongoose.Schema.Types.ObjectId, required: true },
  assetModel: { type: String, enum: ["CarAsset", "EstateAsset", "YachtAsset", "BikeAsset"]
},
  message: { type: String, required: true },
  status: { type: String, default: "New", enum: ["New", "Contacted", "Negotiating",
"Closed"] }
});
```

2.4.2 User Activity Log (UserActivity.model.js)

This tracks behavior for analytics.

```
const userActivitySchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
  activityType: {
    type: String,
    enum: ["VIEW", "CALL_AGENT", "INQUIRY", "BUY_REQUEST", "RENT_REQUEST"]
  }
});
```

```
  },  
  metadata: { type: mongoose.Schema.Types.Mixed } // Stores extra data like "Time spent  
viewing"  
});
```

2.5 Promotional Systems (Coupon.model.js)

Handles discounts for premium plan upgrades.

```
const couponSchema = new mongoose.Schema({  
  code: { type: String, unique: true, uppercase: true }, // e.g., "SUMMER50"  
  discountType: { type: String, enum: ['percentage', 'fixed'] },  
  discountValue: { type: Number, required: true },  
  usageLimit: { type: Number, default: null }, // Prevents exploitation  
  usageCount: { type: Number, default: 0 }  
});  
  
// Custom Method: Built directly into the schema  
couponSchema.methods.isValid = function() {  
  const now = new Date();  
  return this.isActive && this.expiresAt > now && (this.usageLimit === null ||  
this.usageCount < this.usageLimit);  
};
```

2.6 Data Integrity & Middleware

Mongoose isn't just about schemas; it's about **logic**.

- **Timestamps:** Every model uses `{ timestamps: true }`, which adds `createdAt` and `updatedAt` automatically. This allows us to sort by "Newest Listings" effortlessly.
- **Indexing:** Fields like `email` and `googleId` are marked as `unique: true`, creating internal MongoDB indexes for O(1) lookup speeds.
- **Validation:** Enums (like `status`) ensure that no "garbage" data can ever enter the database, keeping the UI clean and predictable.